

银行家算法就是在每次分配给线程新的资源时都进行安全性检查，保证这个分配不会导致系统进入非安全状态。如在表 9.2 所示状态中，假设此时 T_1 发出请求申请资源 A ，申请数量为 1，虽然剩余的资源数量大于这个请求，但是若将 A 资源分配给 T_1 ，系统将无法找到一个可以满足的线程，因此处于非安全状态。此时，系统会安排 T_1 等待，直到其他线程（如 T_2 ）顺利获取资源并释放占有的资源时才再次进行安全性检查。只有在安全性检查通过之后，线程的请求才能够得到满足。

回到本小节开头产生死锁的例子中，我们该如何使用死锁避免算法来消除死锁？这里，我们可以把代码片段 9.17 中的锁 A 与锁 B 都当成仅有 1 份的资源。而线程 A 与线程 B 都需要获取锁 A 与锁 B 。如果线程 A 先获取其中的锁 A ，此时线程 B 向系统申请锁 B ，系统就会检查如果把锁 B 分配给线程 B ，是否会导致系统处于非安全状态。显然，如果此时分配给线程 B ，那么系统将无法找到安全序列。因而系统不会立刻将锁 B 分配给线程 B ，而是待线程 A 结束执行后才分配，从而避免了死锁的产生。

9.3.5 哲学家问题

哲学家问题是一个经典的死锁案例。哲学家问题描述了一个非常有意思的场景：如图 9.9 所示，现在有 5 名哲学家 A 、 B 、 C 、 D 、 E 围着一个圆形餐桌落座。每一位哲学家面前放着一份食物，左手与右手边分别放着一支筷子。每一位哲学家必须同时拿起这两支筷子才能开始进食。然而，筷子数量是有限的：相邻的哲学家中间只有一支筷子。当哲学家感到饥饿时，他将尝试拿起这两支筷子。如果成功获取，则开始进食，并在吃饱后放下筷子；否则将一直拿着已经获取的筷子并等待邻座的哲学家放下

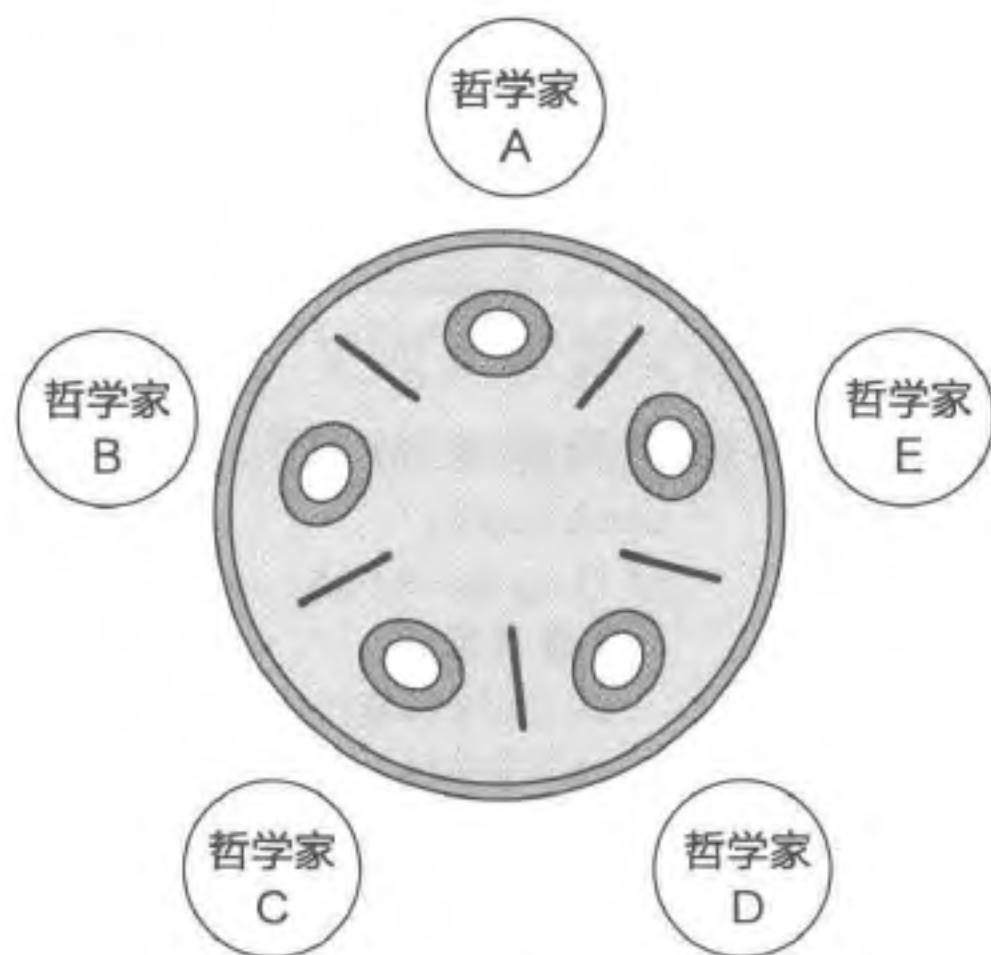


图 9.9 哲学家问题

筷子。现在假设食物是无限的，而哲学家在思考问题的间隙如果感到饥饿就会尝试拿起筷子进食。

在理想的情况下，哲学家可以依次使用筷子，从而都可以吃到食物。然而，存在这样的一种情况：当所有的哲学家都尝试进食且都拿起了他们左手边的筷子时，每一位哲学家都在等待右手边的哲学家放下筷子（如哲学家 A 需要等待哲学家 B 放下筷子）。由于每一位哲学家都不肯让步，没有人会放下已经拿到的筷子，因此这些哲学家永远无法拿到右手边的筷子。此时，我们可以认为发生了死锁：一支筷子只能被一个哲学家使用（互斥访问），哲学家占有了一支筷子并等待另一支（持有并等待），哲学家无法从别人手上抢筷子（资源非抢占），并且形成了哲学家 A 到 B 、 B 到 C 、 C 到 D 、 D 到 E 、 E 又到 A 的循环等待。

9.4 活锁

本节主要知识点

- 活锁与死锁的区别有哪些？
- 如何解决活锁问题？

在介绍死锁预防时，我们提到了一种不允许持有并等待的方法。这种方法能够有效避免死锁，但是会带来一个新的问题：活锁（Livelock）。代码片段 9.19 展示了使用这种策略的具体代码。线程 A 在成功获取锁 A 之后（第 4 行），不同于直接使用阻塞上锁操作拿取锁 B，这里使用 trylock 进行尝试（第 5 行）。trylock 如果成功获取锁，则返回 SUCC，否则返回 FAIL。为了避免持有并等待，一旦获取锁 B 失败，该线程将立刻放弃锁 A（第 11 行），并重新尝试获取锁 A 与锁 B。线程 B 与线程 A 同理。在 9.3.1 节产生死锁的例子中，如果采用如上策略，线程 A 在发现锁 B 上锁之后，会释放锁 A 并重试。因而线程 B 有机会拿到锁 A 并打破死锁。

代码片段 9.19 通过避免持有并等待来避免死锁

```
1 void thread_A(void)
2 {
3     while(TRUE) {
4         lock(&A);
5         if(trylock(&B) == SUCC) {
6             // 临界区
7             unlock(&B);
8             unlock(&A);
9             vbreak;    // 退出循环
10    }
11    unlock(&A);
12 }
13 }
14
15 void thread_B(void)
16 {
17     while(TRUE) {
18         lock(&B);
19         if(trylock(&A) == SUCC) {
20             // 临界区
21             unlock(&A);
22             unlock(&B);
23             break;    // 退出循环
24         }
25         unlock(&B);
26     }
27 }
```


虽然这样能够避免死锁，但是依然存在特殊情况使得没有线程能够同时获取锁 A 与锁 B。考虑这样一种情况：在某一时刻，线程 A 与线程 B 分别获得了锁 A 与锁 B。由于我们采用的策略，线程 A 尝试获取锁 B 失败，便释放了锁 A。同时，线程 B 也获取锁 A 失败，便释放了锁 B。此时，线程 A 与线程 B 又开始了下一轮尝试，并同时分别拿起了锁 A 与锁 B。如此往复，线程 A 与线程 B 都无法同时拿到两把锁进入临界区，此时便出现了活锁。

活锁并没有发生阻塞，而是线程不断重复着“尝试 - 失败 - 尝试 - 失败”的过程，导致在一段时间内没有线程能够成功达成条件并运行下去。由于活锁实际上没有发生阻塞，因此不同于死锁，活锁可能会自行解开。在刚才的例子中，如果两个线程中的一个在再次尝试之前等待了一小段时间（即线程被调度器调度走），另一个线程就有机会一鼓作气拿到两把锁进入临界区。需要注意的是，活锁并非只有在避免死锁时才会产生，这里仅仅使用了避免死锁的场景作为一个例子。对于其他场景，凡是具有类似“尝试 - 失败 - 重试”的模式，都有可能由于多个线程同时希望获取资源，从而相互打断、相互等待，造成活锁的现象。

由于产生的条件比较特殊，并没有一种统一的方法来避免活锁，需要结合具体问题来分析和提出解决方案。如在刚才的例子中，我们可以采取“让线程在获取失败后等待随机的时间再开始下次尝试”的策略来减少出现活锁的概率。又或者要求所有的线程都按照统一的顺序来获取锁。例如让两个线程都按照先拿锁 A 再拿锁 B 的顺序就不会出现活锁。

9.5 思考题

1. 很多数据结构在设计与实现时没有考虑如何保证并发访问时的正确性。而当我们要在多核中使用这些数据结构时，必须修改这些代码来保证线程的安全。代码片段 9.20 给出了一个栈的实现，请分析该实现是否线程安全并解释原因。

代码片段 9.20 栈操作

```
1 typedef struct Node {
2     struct Node *next;
3     int value;
4 } Node;
5
6 void push(Node **top_ptr, Node *n)
7 {
8     n->next = *top_ptr;
9     *top_ptr = n;
10 }
11
12 Node *pop(Node **top_ptr)
```